

TPE 5

ARBRE COUVRANT MINIMAL

Algorithme de Kruskal & Structure Union-Find

VOTRE MISSION

Vous travaillez pour la **SBEE** (Société Béninoise d'Énergie Électrique). Votre mission : implémenter l'algorithme de **Kruskal** pour calculer le réseau électrique de **coût minimal** reliant tous les villages d'une commune.

Ce TPE vous permettra de comprendre comment on optimise les réseaux de câblage, les infrastructures routières, et les réseaux informatiques !

Repository GitHub : [graph-theory-course/session5/tpe/](https://github.com/graph-theory-course/session5/tpe/)

ARBORESCENCE DU PROJET

TPE5_Kruskal/	
include/	Fichiers d'en-tête
graph.h	Classe Graph (fourni)
union_find.h	Classe UnionFind
mst.h	Déclarations MST
src/	Code source
graph.cpp	Implémentation Graph (fourni)
union_find.cpp	À COMPLÉTER (vos TODO)
mst.cpp	À COMPLÉTER (vos TODO)
main.cpp	Programme principal
data/	Graphes de test
graph_test.txt	Petit graphe pour debug
graph_commune.txt	Graphe réaliste commune
tests/	Tests automatisés
test_mst.cpp	16 tests
Makefile	Compilation automatique

LES 7 FONCTIONS À IMPLÉMENTER

Niveau	Fonction	Points	Objectif
BASE	UnionFind(n)	2	Constructeur : créer n ensembles singletons
BASE	find(x)	4	Trouver représentant + compression chemin
BASE	unite(x, y)	4	Fusionner deux ensembles (union par rang)
STANDARD	kruskal()	6	Algorithme complet de Kruskal
STANDARD	print_mst()	2	Afficher le MST joliment
BONUS	prim()	3	Algorithme de Prim (file de priorité)
CODE	Qualité code + commentaires	+2	Noms variables, explications
TOTAL		23	Ramené sur 20 points

DEADLINE : Début de la Séance 6 (Bellman-Ford)

COMPRENDRE KRUSKAL AVANT DE CODER

L'IDÉE CENTRALE

Kruskal trouve le réseau de coût minimal en ajoutant les arêtes **une par une**, de la plus légère à la plus lourde. À chaque étape, on n'ajoute l'arête que si elle ne **crée pas de cycle** (c'est-à-dire si elle relie deux composantes différentes).

C'est un algorithme **GLOUTON** (greedy) : à chaque étape, on choisit l'arête la moins chère. Ce choix localement optimal est aussi globalement optimal grâce à la **propriété de coupe** !

LA CLÉ : UNION-FIND (Disjoint Set Union)

Pour savoir si deux sommets sont dans la même composante, on utilise la structure **Union-Find** :

Pour chaque arête (u, v, poids) triée par poids croissant :

Si `find(u) != find(v)` : // u et v dans des composantes différentes ?

Ajouter (u, v) au MST

`unite(u, v)` // Fusionner les deux composantes

LES STRUCTURES DE DONNÉES

Structure Edge (arête pondérée) :

```
struct Edge {  
    int src, dest, weight;  
};
```

Classe UnionFind :

```
class UnionFind {  
private:  
    vector<int> parent; // parent[i] = parent du sommet i  
    vector<int> rank; // rank[i] = borne sup de la hauteur  
public:  
    UnionFind(int n); // Constructeur  
    int find(int x); // Trouver représentant  
    void unite(int x, int y); // Fusionner deux ensembles  
    bool connected(int x, int y); // Même composante ?  
};
```

DEUX OPTIMISATIONS CRUCIALES

Optimisation	Sans	Avec	Impact
Compression de chemin	$\text{find} = O(n)$	$\text{find} = O(\alpha(n)) \approx O(1)$	find quasi-instant
Union par rang	Arbre dégénéré	Arbre équilibré	Hauteur $O(\log n)$
Les deux ensemble	$O(n)$ par op	$O(\alpha(n)) \approx O(1)$	Presque magique !

COMPLEXITÉ DE KRUSKAL

Étape	Opération	Complexité
1. Tri des arêtes	<code>std::sort</code> sur <code>vector<Edge></code>	$O(E \log E)$
2. Pour chaque arête	<code>find</code> + <code>unite</code>	$O(E \times \alpha(V)) \approx O(E)$
TOTAL		$O(E \log E)$

POURQUOI ÇA MARCHE ?

INVARIANT : Si une arête est l'arête de poids minimum traversant une coupe (séparation du graphe en deux parties), alors elle appartient au MST. Kruskal exploite cet invariant en traitant les arêtes par poids croissant : chaque arête acceptée est forcément la plus légère reliant deux composantes.

GUIDE D'IMPLÉMENTATION

FONCTION 1 : UnionFind(int n) [BASE - 2 points]

Objectif : Constructeur — créer n ensembles singletons (chaque sommet est sa propre composante)

```
UnionFind::UnionFind(int n) {  
    // TODO 1 : Redimensionner parent et rank (taille n)  
    // parent.resize(n) et rank.resize(n)  
    // TODO 2 : Initialiser parent[i] = i pour tout i  
    // (chaque sommet est sa propre racine)  
    // TODO 3 : Initialiser rank[i] = 0 pour tout i  
}
```

FONCTION 2 : find(int x) [BASE - 4 points]

Objectif : Trouver le représentant (racine) avec **compression de chemin**

ATTENTION : La compression est RÉCURSIVE — chaque noeud visité est rattaché directement à la racine !

```
int UnionFind::find(int x) {  
    // TODO 1 : Cas de base : si parent[x] == x, retourner x  
    // TODO 2 : Appel récursif avec compression :  
    // parent[x] = find(parent[x])  
    // (c'est la COMPRESSION DE CHEMIN !)  
    // TODO 3 : Retourner parent[x]  
}
```

FONCTION 3 : unite(int x, int y) [BASE - 4 points]

Objectif : Fusionner deux ensembles avec **union par rang**

```
void UnionFind::unite(int x, int y) {  
    // TODO 1 : Trouver les racines rx = find(x) et ry = find(y)  
    // TODO 2 : Si rx == ry, ne rien faire (même ensemble, return)  
    // TODO 3 : Union par rang :  
    // Si rank[rx] < rank[ry] : parent[rx] = ry  
    // Si rank[rx] > rank[ry] : parent[ry] = rx  
    // Si égaux : parent[ry] = rx ET rank[rx]++  
}
```

FONCTION FOURNIE : connected(int x, int y)

```
// Déjà implémentée – utilise find() que VOUS avez codé  
bool UnionFind::connected(int x, int y) {  
    return find(x) == find(y);  
}
```

FONCTION 4 : kruskal() [STANDARD - 6 points]

Objectif : Algorithme complet de Kruskal — C'EST LA FONCTION PRINCIPALE !

Complexité : $O(E \log E)$

```
vector<Edge> kruskal(Graph& g, int& total_weight) {  
    // TODO 1 : Extraire toutes les arêtes dans un vector<Edge>  
    // Attention : graphe non orienté → ne prendre chaque arête qu'une fois !  
    // TODO 2 : Trier par poids croissant avec std::sort  
    // sort(edges.begin(), edges.end(), [](Edge& a, Edge& b) {  
    // return a.weight < b.weight; });  
    // TODO 3 : Créer un UnionFind pour V sommets  
    // TODO 4 : Boucle sur les arêtes triées :  
    // Pour chaque Edge e :
```

```

// Si !uf.connected(e.src, e.dest) :
// Ajouter e au vector<Edge> mst
// uf.unite(e.src, e.dest)
// total_weight += e.weight
// TODO 5 : Arrêter si mst.size() == V - 1 (optim)
// TODO 6 : Retourner le vector mst
}

```

FONCTION 5 : print_mst() [STANDARD - 2 points]

```

void print_mst(const vector<Edge>& mst, int total_weight) {
// TODO 1 : Parcourir le vector et afficher chaque arête :
// cout << e.src << " -- " << e.dest
// << " (poids " << e.weight << ")" << endl;
// TODO 2 : Afficher le poids total à la fin
}

```

FONCTION 6 (BONUS) : prim() [BONUS - 3 points]

Objectif : Implémenter Prim comme alternative — similaire à votre Dijkstra du TPE4 !

```

vector<Edge> prim(Graph& g, int& total_weight) {
// Similaire à Dijkstra : faire pousser l'arbre depuis le sommet 0
// Utiliser un vector<int> key(V, INT_MAX) (poids min pour rejoindre l'arbre)
// priority_queue<pair<int,int>, vector<pair<int,int>>,
// greater<pair<int,int>>> pq; // Min-Heap !
// À chaque itération : extraire le sommet avec key[] minimum
}

```

PIÈGES À ÉVITER ABSOLUMENT

PIÈGE #1 : Oublier l'initialisation `parent[i] = i`

Symptôme : `find()` retourne toujours 0 ou des valeurs incorrectes

Cause : `parent[]` est initialisé à 0 par défaut (au lieu de `i`)

```
// ■ CORRECT
for (int i = 0; i < n; i++) {
    parent[i] = i; // Chaque sommet est sa propre racine !
    rank[i] = 0;
}
```

PIÈGE #2 : `find()` sans compression de chemin

Symptôme : Programme très lent sur grands graphes

```
// ■ FAUX (pas de compression –  $O(n)$  par appel)
int UnionFind::find(int x) {
    while (parent[x] != x) x = parent[x];
    return x;
}

// ■ CORRECT (avec compression –  $O(\alpha(n)) \approx O(1)$  amorti)
int UnionFind::find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]); // Compression !
    return parent[x];
}
```

PIÈGE #3 : Oublier le cas `rx == ry` dans `unite()`

Symptôme : Le MST a trop d'arêtes ou des cycles

```
void UnionFind::unite(int x, int y) {
    int rx = find(x);
    int ry = find(y);
    if (rx == ry) return; // ← NE PAS OUBLIER !
    // ... union par rang
}
```

PIÈGE #4 : Mauvais tri avec `std::sort`

Symptôme : Les arêtes ne sont pas triées → MST incorrect

```
// ■ FAUX (tri décroissant !)
sort(edges.begin(), edges.end(), [](Edge& a, Edge& b) {
    return a.weight > b.weight; // Descending !
});

// ■ CORRECT (tri croissant par poids)
sort(edges.begin(), edges.end(), [](const Edge& a, const Edge& b) {
    return a.weight < b.weight; // Ascending !
});
```

PIÈGE #5 : Compter les arêtes deux fois (graphe non orienté)

Symptôme : Le vector d'arêtes a des doublons → résultat faux ou lent

```
// ■ Ne prendre chaque arête qu'une fois
// Si le graphe stocke les deux sens ( $u \rightarrow v$  et  $v \rightarrow u$ ), filtrer :
for (int u = 0; u < V; u++)
    for (auto& [v, w] : g.adj[u])
        if (u < v) // Ne garder que dans un sens
            edges.push_back({u, v, w});
```

PIÈGE #6 : Oublier l'arrêt à $n-1$ arêtes

Symptôme : Performances inutilement mauvaises sur graphes denses

```
// ■ Arrêt anticipé : un MST a exactement  $V-1$  arêtes  
if ((int)mst.size() == V - 1) break; // Toutes les arêtes trouvées !
```

TESTS, BARÈME ET RENDU

GRAPHES DE TEST

1. graph_test.txt (7 sommets, 11 arêtes)

→ Même graphe que l'exercice 1 du TD5 — facile à vérifier à la main

→ MST attendu : poids total = 28

2. graph_commune.txt (10 villages, 18 routes)

→ Graphe réaliste : réseau de villages d'une commune

→ Utilisé pour la fonction bonus (Prim)

COMPILATION ET TESTS

Compiler : make

Exécuter : ./bin/main

Tests : make test && ./bin/test_mst

Sortie : 16 tests réussis sur 16

BARÈME DÉTAILLÉ

Fonction	Tests	Points	Critères clés
UnionFind(n)	2	2	resize, parent[i]=i, rank[i]=0
find(x)	3	4	Récuratif, compression de chemin
unite(x, y)	3	4	Union par rang, cas rx==ry
kruskal()	4	6	sort, boucle find+unite, arrêt V-1
print_mst()	2	2	Format lisible, poids total
prim()	2	3	priority_queue, visited[], key[]
Qualité code	—	+2	Noms, commentaires, indentation
TOTAL	16	23	Ramené sur 20 points

FORMAT DE RENDU

Fichier à soumettre : **NOM_Prenom_TPE5.zip**

Contenu du ZIP :

- src/union_find.cpp (votre implémentation Union-Find)
- src/mst.cpp (votre implémentation Kruskal)
- capture_tests.png (screenshot des 16 tests réussis)

NE PAS INCLURE : fichiers .o, bin/, obj/

CHECKLIST AVANT DE RENDRE

- Les 5 fonctions de base sont implémentées (+ bonus optionnel)
- make compile sans erreurs ni warnings (-Wall -Werror)
- ./bin/test_mst affiche 16/16 tests réussis
- Le code est commenté et bien indenté
- Les noms de variables sont explicites (pas de a, b, x1...)
- La complexité de chaque fonction est documentée en commentaire
- Le fichier ZIP contient union_find.cpp + mst.cpp + capture
- Le nom du ZIP respecte le format NOM_Prenom_TPE5.zip

CONSEILS POUR RÉUSSIR

1. Commencez par le **constructeur** → **find** → **unite** (testez après chaque fonction !)

2. Ensuite seulement **kruskal()** — il utilise les 3 méthodes précédentes
3. Vérifiez votre MST à la main avec le graphe du TD5
4. Relisez les **6 pièges** de la page 4 avant de coder
5. Utilisez `cout` pour debugger : affichez `parent[]` après chaque `unite()`
6. Vérifiez que votre MST a exactement **V-1 arêtes**
7. Pour le bonus Prim, inspirez-vous de votre **dijkstra_heap()** du TPE4 !
8. N'hésitez pas à demander de l'aide si vous êtes bloqué !

Bon courage !

Kruskal et Union-Find sont des structures fondamentales en informatique.

On les retrouve partout : réseaux, clustering, segmentation d'images, bioinformatique...